

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester III)
Scala For Data Science

Practical XXVII

Roll No.: S021	Name: LODHI IQRA
Class: SYBSc IT	Subject: Scala For Data Science
Date of Assignment: 21/08/2026	Date/Time of Submission: 01/09/2026

AIM: Implement one-hot encoding for a categorical column in a dataset.

CODE:

```
build.sbt  IQRA.scala x
1  > object Demo extends App
2    {
3      val data = List("Red", "Blue", "Green", "Red", "Blue")
4      for (value <- data)
5      {
6        val r = if (value == "Red") 1 else 0
7        val b = if (value == "Blue") 1 else 0
8        val g = if (value == "Green") 1 else 0
9        println(value + " -> Red:" + r + " Blue:" + b + " Green:" + g)
10     }
11
12     println("IQRA S021")
13   }
14
15
```

OUTPUT:

```
↑ "C:\Program Files\Java\jdk1.8.0_18
↓ Red -> Red:1 Blue:0 Green:0
⇌ Blue -> Red:0 Blue:1 Green:0
⇌ Green -> Red:0 Blue:0 Green:1
⇌ Red -> Red:1 Blue:0 Green:0
⇌ Blue -> Red:0 Blue:1 Green:0
⇌ IQRA S021
↓
```

Practical XXVIII

AIM: Compute the Euclidean distance between two Breeze vectors. Use it for nearest neighbor classification

CODE:

```
1 import breeze.linalg._
2 object IQRA extends App
3 {
4     val A = DenseVector(2.0, 3.0)
5     val B = DenseVector(5.0, 7.0)
6     val distance = norm(A - B)
7
8     println("Euclidean Distance: " + distance)
9     val C = DenseVector(2.5, 3.5)
10    val d1 = norm(C - A)
11    val d2 = norm(C - B)
12
13    if (d1 < d2)
14        println("Nearest Neighbor: A")
15    else
16        println("Nearest Neighbor: B")
17    println("IQRA S021")
18 }
19
20
21
```

OUTPUT:

```
"C:\Program Files\Java\jdk1.8.0_181\bin\  
Euclidean Distance: 5.0  
Nearest Neighbor: A  
IQRA S021  
  
Process finished with exit code 0
```

Practical XXIX

AIM: Create a simple Spark MLlib pipeline to classify data.
Use logistic regression or decision trees.

CODE:

```
1 import org.apache.spark.sql.SparkSession
2 import org.apache.spark.ml.feature.VectorAssembler
3 import org.apache.spark.ml.classification.LogisticRegression
4
5 object Demo extends App {
6
7     val spark = SparkSession.builder()
8         .appName("Classification")
9         .master("local[*]")
10        .getOrCreate()
11
12    import spark.implicits._
13
14    val data = Seq(
15        (1.0, 0.0),
16        (2.0, 0.0),
17        (3.0, 0.0),
18        (7.0, 1.0),
19        (8.0, 1.0),
20        (9.0, 1.0)
21    ).toDF("marks", "label")
22
23    val a = new VectorAssembler()
24        .setInputCols(Array("marks"))
25        .setOutputCol("features")
26
27    val d = a.transform(data)
28
29    val model = new LogisticRegression()
30        .setLabelCol("label")
31        .setFeaturesCol("features")
```

```
val data = Seq(
  (1.0, 0.0),
  (2.0, 0.0),
  (3.0, 0.0),
  (7.0, 1.0),
  (8.0, 1.0),
  (9.0, 1.0)
).toDF("marks", "label")

val a = new VectorAssembler()
  .setInputCols(Array("marks"))
  .setOutputCol("features")

val d = a.transform(data)

val model = new LogisticRegression()
  .setLabelCol("label")
  .setFeaturesCol("features")

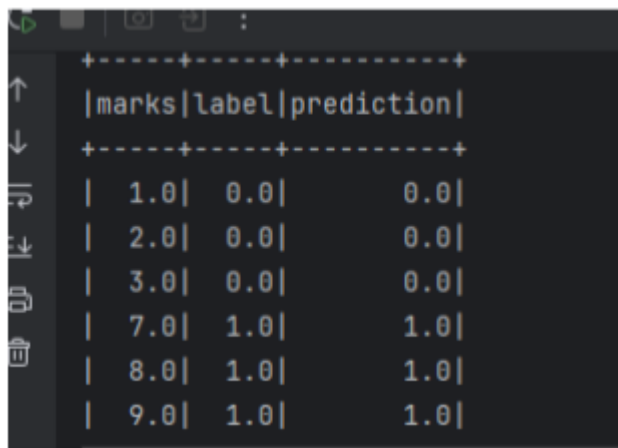
val result = model.fit(d)

val prediction = result.transform(d)

prediction.select("marks", "label", "prediction").show()

spark.stop()
```

OUTPUT:



A terminal window with a dark background and light gray text. The window title bar at the top shows a green icon, a gray icon, and a colon. On the left side of the terminal, there is a vertical toolbar with several icons: a right arrow, a left arrow, a double left arrow, a double right arrow, a copy icon, and a paste icon. The main content of the terminal is a table with three columns: 'marks', 'label', and 'prediction'. The table is enclosed in a border made of dashed lines and solid lines. The data rows are as follows:

marks	label	prediction
1.0	0.0	0.0
2.0	0.0	0.0
3.0	0.0	0.0
7.0	1.0	1.0
8.0	1.0	1.0
9.0	1.0	1.0